SOURCE-PINNED TECHNICAL REVIEW

AsymptoticAnalysis

Defining-sum integration, request semantics, and an integer-indexed Dirichlet algebra

New findings after the existing review corpus;
focused repairs, mathematical contracts, and executable artifacts

Prepared for Vladimir Reshetnikov
10 September 2026

Repository

`https://github.com/VladimirReshetnikov/Asymptotic`

Reviewed commit

`8cee870994f506b501bae3ea6bd4a3a7edb895c1`

Evidence in this review. Selected public results and a three-edit candidate were executed in Wolfram Language 15.0.0. The independent Python artifacts pass 32 test methods. Mathics compatibility was examined in source, but no Mathics runtime was available. Neither the full upstream suite nor the complete packaged Wolfram probe file was run. The two new findings are coverage and request-contract issues; neither is presented as a newly discovered false asymptotic theorem.

Abstract

This audit examined the canonical forward and inverse machinery, series operations, exact interval primitives, several special-function modules, and Mathics compatibility layers, against the repository's extensive existing review record. Most plausible candidates encountered in those areas were already documented, already guarded, or not substantiated. They are not reintroduced as findings.

Two reproducible additions remain in the defining-sum adapter. First, the package constructor accepts bare large-argument Zeta and Lerch expressions, but rejects simple affine translations of those expressions even though its existing series arithmetic can construct the corresponding results. Second, the same adapter ignores an explicitly supplied `SeriesTermGoal` whenever an explicit cutoff is present. Its result can record a requested goal of one and a returned count of three. In the Zeta path this also causes a resource refusal for a request whose term goal makes the expensive work unnecessary.

The article isolates both mechanisms, supplies a narrowly scoped term-goal patch and a restricted affine-lifting prototype, and develops an additional integer-indexed Dirichlet-jet prototype. The latter implements exact coefficient addition, convolution, reciprocal prefixes, and elementary global coefficient majorants. Its purpose is a concrete development option for this particular family, rather than another proposal for an unrestricted transseries framework. Native observations, independent tests, candidate mechanisms, and unexecuted artifacts are recorded separately throughout.

Contents

1 Assessment, scope, and nonduplication

1.1 What should change first

The smallest useful upstream change is to make the defining-sum adapter obey both explicit request constraints. It requires three local edits, preserves the existing coefficient and remainder formulas, and has successful focused Wolfram controls. The next change should make rational affine wrappers around an admitted defining-sum atom reach that atom's constructor, rather than falling through to a native coefficient importer that cannot interpret it.

These are deliberately modest priorities. The retained review corpus already contains substantial correctness, performance, domain, precision, and architecture recommendations. Recommending those same projects again would obscure the incremental contribution requested here. [1, 2]

ID	Classification	Finding	Evidence
N01	Medium; coverage and UX	Bare Zeta/Lerch atoms are admitted, but their simple affine translations are refused by the public Package constructor. Existing arithmetic works around the dispatch gap.	Four native constructor observations; two existing-arithmetic controls.
N02	Medium; request semantics and work selection	Explicit cutoffs disable the term goal in both defining-sum constructors. Zeta can reject a cheap one-term request on the basis of work it need not perform.	Native count mismatches and resource refusal; five candidate controls.
D01	Development option, not a defect	Integer-indexed Dirichlet jets with exact convolution and reciprocal coefficient bounds.	Implemented independent Python prototype; mathematical derivation and tests.

The severity labels concern ordinary correctness of the public request contract and usefulness of the API. N01 returns a failure rather than a wrong expression. N02 returns a mathematically valid longer expansion, but does not honor the requested term count. No security severity or comprehensive acceptance claim is implied.

1.2 Snapshot and retrieval boundary

The review is pinned to commit `8cee870994f506b501bae3ea6bd4a3a7edb895c1`, not to an unspecified moving `main`. The root standalone was imported as text and then loaded from a local temporary file in the Wolfram evaluator. The import contained 692,380 characters. A smoke call for `Sin[x]` at cutoff three returned a `GeneralizedSeries` with normal expression x, consistent with the package's exclusive cutoff convention. [6]

The substantive source inspection covered 21 canonical modules to different depths. These include the main kernel, ordinary and composite arithmetic, series operations, Dirichlet special functions, Gamma/Barnes forward paths, selected inverse operations, Fourier coefficients, interval certificates, and six Mathics compatibility modules. The coverage record in `evidence/source-coverage.json` distinguishes full returned modules from selected or overlapping ranges. This is substantial targeted analysis, not a claim that every source line, test, script, or historical document received an equal audit.

1.3 How earlier findings were excluded

At the pinned snapshot, the review index contains 35 retained packages across five waves, plus ten retirement tombstones. The consolidated status register, all five wave indexes, and selected original discussions were used as the primary suppression baseline. A further keyword-and-context scan read 62 TeX files from the retained articles, including split sections and standalone copies. The scan for waves one through three explicitly reported 47 reads and no unread files; the remaining fifteen files belong to waves four and five. This is not a count of 62 independent reviews. [1, 3, 4, 2]

The lexical scan looked for Dirichlet, Zeta, Lerch, divisor, and multiplicative algebra terminology and examined the returned contexts. It supplements, rather than replaces, semantic comparison with the maintained finding summaries. It cannot prove the absence of an equivalent idea expressed with different words. The new entries below therefore include their nearest related earlier entries and explain the difference. The machine-readable ledger is `evidence/novelty-ledger.json`.

One public wrong-result candidate encountered during the investigation overlapped an earlier sided-germ obligation. It was suppressed rather than used to inflate the new-finding count. The same rule was applied to previously recorded certificate, truncation, native-boundary, and general budget topics. Their reproduction details and recommendations are not repeated here.

1.4 Execution evidence and its limits

Evidence population	What was obtained	What it does not establish
Wolfram baseline	Public constructor, term-count, resource-refusal, and arithmetic observations on version 15.0.0 for Linux x86-64, build dated 6 May 2026.	Full upstream-suite acceptance or equivalent behavior on every supported platform.
Term-goal candidate	Three guarded edits applied to the imported standalone; five focused outputs including unchanged no-goal controls.	Full rebuilt modular distribution acceptance, combined changes, or Mathics execution.
Independent Python	28 arithmetic/majorant test methods and four synthetic patch-anchor fixture methods; all 32 passed.	Execution of the Wolfram package or proof of an arbitrary caller-declared infinite coefficient bound.
Affine prototype	Packaged standalone helper and a probe driver; the underlying existing arithmetic operations succeeded in separate native calls.	Successful execution of the complete helper file. Its grouped mechanism probe failed at the connector boundary.
Mathics	Source-level examination of the compatibility layers and the vocabulary used by the proposed changes.	A Mathics baseline, patched run, benchmark, or compatibility certification.

Several requests to the remote evaluator failed with transport errors. A direct remote `Get` also received a truncated source stream; importing the complete text and loading a local file succeeded. These events are recorded as execution infrastructure limitations, not repository syntax or algorithm defects.

2 Where the new issues occur

2.1 Two special-function inputs, two natural coordinates

The defining-sum module supplies constructions unavailable to a straightforward ordinary Taylor path. For large real S, the Zeta adapter uses the convergent sum

$$\zeta(S) = \sum_{n=1}^{\infty} n^{-S}, \qquad S > 1, \tag{1}$$

and chooses $w = e^{-S}$. Its exact weights are $\log n$, since $n^{-S} = w^{\log n}$. An exclusive cutoff h retains precisely the terms with $\log n < h$. The $n = 1$ term is the constant one and is counted as a term by this adapter. [7, 16]

The Lerch adapter instead treats fixed real parameters z, s with $|z| < 1$ and an argument A tending to positive infinity. It uses $w = 1/A$, with blocks of weights $s + k$ and coefficients

$$c_k = \frac{(-1)^k (s)_k}{k!} M_k(z), \qquad M_0(z) = \frac{1}{1-z}, \quad M_k(z) = \mathrm{Li}_{-k}(z) \ (k > 0). \tag{2}$$

The defining series is $\Phi(z, s, A) = \sum_{n \geq 0} z^n (A + n)^{-s}$ in the admitted real domain. For the witness $z = 1/2, s = 1$, the first coefficients are $2, -2, 6, -26, \ldots$. [7, 17]

The original module already attaches explicit remainder metadata to these constructions. This article does not re-present the module's existing tail bounds as a new result or a newly discovered need for majorants. The findings are in admission and request selection around that coefficient machinery.

2.2 The constructor and the arithmetic layer have different entry points

The important boundary is not between a "capable Wolfram" and an "incapable Mathics." The source contains a dedicated dispatcher, `dirichletSpecialForwardExpansion`, whose initial guard admits only a whole expression matching

```
Zeta[_] | LerchPhi[_, _, _]
```

An affine wrapper has head `Plus` or `Times`, so this guard declines it. Subsequent generic routes do not automatically convert its special-function leaf into the result that the defining-sum adapter would have returned. That is the mechanism behind N01. [7]

The ordinary arithmetic layer is a separate consumer of already constructed `GeneralizedSeries` objects. It has guarded routes for regular operands, compatible ordered series, and composite envelopes. Consequently, the fact that a wrapped expression fails during construction does not imply that its sum cannot be represented after the leaf is constructed. [8, 10, 9]

This distinction suggests a local integration change: make a small admitted expression grammar construct its special leaf and then reuse the existing arithmetic. It does not require changing the mathematical definition of Zeta, modifying native `Series`, or admitting all expressions containing special functions.

3 N01: an admitted atom becomes unsupported when translated

New coverage/UX finding; successful Wolfram baseline observations.
Location: the whole-expression guard in `dirichletSpecialForwardExpansion`, in `src/Kernel/DirichletSpecialFunctions.wl`.
Boundary: explicit `"Backend"->"Package"`. Automatic/native behavior outside the observed calls is not inferred.

3.1 Public witnesses

With the pinned package loaded, the following call succeeds:

```
AsymptoticAnalysis`AsymptoticExpansion[
  Zeta[x], {x, Infinity, 2}, "Backend" -> "Package"]
```

Its normal expression is

$$1 + 2^{-x} + 3^{-x} + 4^{-x} + 5^{-x} + 6^{-x} + 7^{-x}, \tag{3}$$

a nonzero remainder being recorded as `PowerLogRemainder[E^(-x),Log[8],0]`. The translated input

```
AsymptoticAnalysis`AsymptoticExpansion[
  Zeta[x] - 1, {x, Infinity, 2}, "Backend" -> "Package"]
```

returns `Failure["UnsupportedNativeCoefficient",...]` instead. The reported unsupported coefficient contains a retained `Zeta[u$12^(-1)]`; the generated local symbol is incidental. The message asks for polynomial logarithmic coefficients and bounded real sine/cosine modes. It does not tell the caller that the nearby bare expression has a successful defining-sum constructor.

The Lerch pair is equally simple:

```
AsymptoticAnalysis`AsymptoticExpansion[
  LerchPhi[1/2, 1, x], {x, Infinity, 2},
  "Backend" -> "Package"]
(* Normal: 2/x; remainder: O(x^-2) *)

AsymptoticAnalysis`AsymptoticExpansion[
  1 + LerchPhi[1/2, 1, x], {x, Infinity, 2},
  "Backend" -> "Package"]
(* Failure["UnsupportedNativeCoefficient", ...] *)
```

Both successful and failed outputs are transcribed in `evidence/native-observations.json`. This is a failure to integrate an existing capability, not evidence that the underlying defining sum or its remainder bound is false.

3.2 An existing public workaround

The package's own arithmetic provides a useful control:

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  Zeta[x], {x, Infinity, Log[3]}, "Backend" -> "Package"];
```

```
AsymptoticAnalysis`SeriesAdd[s, -1]
(* GeneralizedSeries; Normal: (E^(-x))^Log[2] *)

s = AsymptoticAnalysis`AsymptoticExpansion[
  LerchPhi[1/2, 1, x], {x, Infinity, 3},
  "Backend" -> "Package"];
AsymptoticAnalysis`SeriesAdd[s, 1]
(* GeneralizedSeries; Normal: 1 - 2/x^2 + 2/x *)
```

The first normal expression equals 2^{-x} on the recorded real approach. These controls demonstrate that the representation and arithmetic need not be extended to handle the two translations. The missing step is reaching them from the ordinary expression constructor.

The grouped arithmetic observation also emitted repeated native `Power::infy` and `Greater::nord` messages. Their originating subcall was not isolated. They are retained in the evidence record but are not elevated to a separate defect or explained by a speculative mechanism.

3.3 Why this is distinct from the earlier Lerch chart finding

Report 24's N03 concerns an already constructed reciprocal-square Lerch chart: recovering a signed coordinate such as $1/x^2$ can lose ordered arithmetic. The current Lerch witness uses the direct argument x, and its existing arithmetic control succeeds. The failure is earlier, at the construction of `1+LerchPhi[...]` as an ordinary expression. A signed quadratic chart repair therefore does not address it. [5]

Similarly, this is not another request for universal expression closure. A rational affine wrapper is a narrow, explicitly checkable grammar whose admitted examples require no new coefficient algebra. More general products, multiple atoms, parameter-dependent factors, or nested functions should remain separate admission decisions.

3.4 A restricted lifting rule

For a rational constant pair b, c, one varying admitted atom $A(x)$, and a series result $S = e + O(R)$ for that atom, the desired transformation is simply

$$c + bA(x) = c + be(x) + O(|b|R(x)). \tag{4}$$

The package already implements the corresponding scalar multiplication and addition. The prototype in `code/AffineDirichletExpansion.wl` performs these steps, then applies the requested cutoff using the existing truncation operation.

The prototype deliberately requires exactly one varying Zeta/Lerch atom, degree-one dependence on that atom, and rational constant affine coefficients. It declines quadratic dependence, inexact coefficients, variable-dependent multipliers, and multiple distinct atoms. It adds one symbol under `AsymptoticAudit`` and does not change any upstream definition or install System upvalues.

```
Get["AsymptoticAnalysis.wl"];
Get["AffineDirichletExpansion.wl"];

AsymptoticAudit`AffineDirichletExpansion[
  Zeta[x] - 1, {x, Infinity, Log[3]}]

AsymptoticAudit`AffineDirichletExpansion[
  1 + LerchPhi[1/2, 1, x], {x, Infinity, 3}]
```

The cutoff in this interface belongs to the special atom's coordinate. In particular, a Zeta cutoff is not silently reinterpreted as a power of $1/x$. The prototype promises an asymptotic remainder carried through the existing arithmetic, not retention of every quantitative bound attached to the source atom. It does not expose a term-goal option.

3.5 The term-goal trap when integrating the rule upstream

A direct implementation must count blocks of the final normalized expression, not merely blocks requested from the atom. For example, if the user asks for one term of $\zeta(x) - 1$, asking the bare Zeta constructor for one term yields only the constant one; subtracting one then erases the only retained block. The desired leading nonzero block is 2^{-x}.

For the rational affine grammar, the constant addition can cancel or introduce at most one weight-zero block. A limited implementation can therefore request one extra source block when a term goal is active, merge the affine constant, and then apply the final goal. It must retain the primitive's honest remainder when a cutoff prevents additional source information. This is a specific one-block accounting rule for affine lifting, not a general backward-precision planner.

The standalone prototype avoids this extra policy by accepting an explicit cutoff only. The integrated constructor should eventually support both constraints, but it should do so after N02's primitive request handling is fixed.

4 N02: explicit cutoffs disable the term goal

New request-contract finding; successful baseline and candidate observations.
Locations: the cutoff branch of `dirichletZetaForward` and the stopping predicate in `dirichletLerchForward`.
Effect: extra returned blocks and, for Zeta, avoidable resource refusal. The returned longer series is not mathematically false.

4.1 Contradictory request and result metadata

The ordinary constructor already applies an integer term goal after the exclusive cutoff: `makeForwardObject` takes at most the requested number of retained blocks. The defining-sum constructors bypass that finishing path and implement their own selection. [6, 7]

The Lerch witness is:

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  LerchPhi[1/2, 1, x], {x, Infinity, 4},
  SeriesTermGoal -> 1, "Backend" -> "Package"];
{Normal[s], s["ReturnedTermCount"], s["RequestedTermGoal"]}
(* {6/x^3 - 2/x^2 + 2/x, 3, 1} *)
```

With the same cutoff and goal, the ordinary input `1+1/x+1/x^2` returns only the constant one, records a returned count of one, and places the first omitted power at x^{-1}. Thus the mismatch is observable within the same package API, not a disputed inference from native Wolfram option behavior.

For Zeta the corresponding observation is:

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  Zeta[x], {x, Infinity, Log[4]},
  SeriesTermGoal -> 1, "Backend" -> "Package"];
{Normal[s], s["ReturnedTermCount"], s["RequestedTermGoal"]}
(* {1 + 2^(-x) + 3^(-x), 3, 1} *)
```

The expectation is to retain one admitted nonzero block, here the constant one, and to report a remainder beginning at 2^{-x}. The public usage of `SeriesTermGoal` as a term-count control is also consistent with the native option's documented purpose, but the decisive evidence is the repository's own ordinary constructor and recorded metadata. [15, 6]

4.2 The exact source mechanism

Zeta distinguishes an automatic-cutoff request from an explicit-cutoff request. In the automatic case it sets `count=goal`. In the explicit case it binary-searches all integers whose logarithms lie below the cutoff and never caps the result by the goal.

The Lerch path makes the exclusivity equally visible:

```
If[If[cut === Automatic,
     Length[rows] >= goal,
     ! less[s + k, cut]],
   frontier = {s + k, coefficient, k}; Break[]];
```

The goal and cutoff are alternatives in that predicate. They should instead be two independent reasons to stop retaining the next nonzero block. Validation already accepts a positive integer goal together with an explicit exact cutoff, so this is not an unsupported argument combination rejected by design. [7]

4.3 A small term goal must constrain preflight work too

A post hoc truncation alone is not enough for Zeta. The baseline also executes

```
AsymptoticAnalysis`AsymptoticExpansion[
  Zeta[x], {x, Infinity, 1000000},
  SeriesTermGoal -> 1, "MaxTerms" -> 100,
  "Backend" -> "Package"]
(* Failure["ResourceLimit", ...] *)
```

Its resource message says that the exponential-coordinate cutoff exceeds `MaxTerms`. Under the combined request semantics, however, the only retained block is one, and the first omitted integer is two. No enumeration near $e^{1000000}$, nor even a 100-term expansion, is needed.

The candidate therefore restricts the binary-search interval itself. When the goal is a positive integer g, the upper search sentinel becomes $\min(g, L) + 1$, where L is the existing term budget. A cutoff-based resource refusal is necessary only when the cutoff would require more than L terms *and* no active goal reduces the retained count to at most L.

4.4 The three-edit candidate

The staged patch changes exactly these expressions:

```
(* Zeta preflight: old condition *)
less[Log[limit + 1], cut]

(* New condition *)
less[Log[limit + 1], cut] &&
  (goal === Automatic || goal > limit)

(* Zeta search interval *)
low = 0;
high = If[goal === Automatic,
  limit + 1, Min[limit, goal] + 1];

(* Lerch stopping rule, after a nonzero coefficient is found *)
(cut =!= Automatic && ! less[s + k, cut]) ||
  (goal =!= Automatic && Length[rows] >= goal)
```

All coefficient formulas, existing domain tests, and bound constructors remain unchanged. In particular, the Lerch loop still finds a *nonzero* frontier coefficient before stopping; it does not count vanished moment coefficients as returned terms.

4.5 Why the selected frontier remains correct

For Zeta, let $m(h)$ be the number of positive integers with $\log n < h$. With both constraints active, the retained count is

$$N = \min\{m(h), g\}. \tag{5}$$

The new binary search computes this minimum without needing $m(h)$ when it is larger than the goal. The first omitted integer is still $N + 1$, so the existing remainder constructor can use $\rho = \log(N + 1)$ and its existing tail metadata. The proof does not depend on approximating e^h numerically. Exact comparisons continue to decide logarithmic boundary equality.

For Lerch, suppose the nonzero coefficient indices are $k_0 < k_1 < \cdots$. The loop retains the first indices whose weights $s + k_j$ lie below the cutoff, stopping also when g such blocks have been retained. The next nonzero coefficient supplies the same kind of frontier as before. Only the selected frontier changes; the existing Taylor-moment bound is evaluated at that frontier's index.

4.6 Observed candidate results

The three anchor strings were each verified to occur exactly once in the imported pinned standalone. After replacement, that temporary file was loaded in a fresh Wolfram evaluator. The following outputs were observed, with `MaxTerms->100`:

Input	Cutoff	Goal	Candidate normal expression and remainder
$\zeta(x)$	$\log 4$	1	$1 + O(2^{-x})$; returned count 1.
$\Phi(1/2, 1, x)$	4	1	$2/x + O(x^{-2})$; returned count 1.
$\zeta(x)$	10^6	1	$1 + O(2^{-x})$; no resource refusal.
$\zeta(x)$	$\log 4$	Automatic	$1 + 2^{-x} + 3^{-x} + O(4^{-x})$; unchanged.
$\Phi(1/2, 1, x)$	4	Automatic	$2/x - 2/x^2 + 6/x^3 + O(x^{-4})$; unchanged.

The candidate changes achieved precision when it honors the smaller term goal; it does not claim that a one-term result has the longer expansion's remainder. That distinction is essential. Merely deleting two terms while retaining the old fourth-order error would introduce a mathematical error that the baseline does not have.

4.7 Novelty and integration boundary

Earlier reports discuss configured backend defaults and special inverse power cutoffs. N02 is not a repeat of those dispatch/default bugs: the options in these witnesses are explicit, already parsed, and accurately stored in the result. The two defining-sum loops then fail to apply them together. The patch is local to those loops rather than another general request-normalization proposal. [3, 4]

The Python staging utility checks all three anchors before writing anything, refuses to overwrite existing output, and writes a canonical module plus a unified diff into a separate directory. It does not modify the checkout in place. The successful native experiment edited the generated standalone's corresponding source text; a canonical-module edit followed by the upstream builder remains a separate integration step to execute before adoption.

5 A narrow development option: integer-indexed Dirichlet jets

5.1 The representation opportunity

The Zeta adapter currently exposes the sequence $\log 1, \log 2, \log 3, \ldots$ as ordinary exact real weights. That is mathematically appropriate for its positive coordinate. But an algebra dedicated to this defining-sum family can retain the *integer index* before converting to logarithmic weights:

$$A(S) = \sum_{n \geq 1} a_n n^{-S}. \tag{6}$$

For positive integer indices, order and multiplication reduce to

$$\log n < \log m \iff n < m, \qquad \log n + \log m = \log(nm). \tag{7}$$

Thus this family has a more specific algebra than arbitrary real exponent weights. Its coefficients can be combined using integer multiplication and divisor relations. This is not a claim that the current generic jet algebra cannot express the same functions, nor a measured claim that it is slow. It is a concrete optional specialization.

The supplied independent prototype restricts coefficients to rational numbers and the growth exponent to a nonnegative integer. Those restrictions make coefficient operations, majorant selection, and tail evaluation at integer S exact using only the Python standard library. A future Wolfram implementation could use a wider exact coefficient ring without changing the indexing idea.

5.2 The prefix invariant

A jet records a positive integer N, all coefficients for $1 \leq n \leq N$, and a global coefficient bound

$$|a_n| \leq C n^p \quad (n \geq 1), \qquad C \geq 0, \quad p \in \mathbb{Z}_{\geq 0}. \tag{8}$$

The coefficient dictionary is sparse, but its *known prefix is dense information*: an absent dictionary entry inside $1, \ldots, N$ means a known zero. An absent entry beyond N is unknown, not zero. The accessor refuses such an out-of-prefix query.

The public constructor declares the infinite bound and checks it on every supplied prefix coefficient. That finite check cannot establish an arbitrary infinite assertion. The Zeta factory establishes its bound by construction

because every coefficient is one. The finite-polynomial factory likewise knows its entire sequence. Arithmetic methods then transport a supplied bound by the propositions below. The distinction is part of the class contract, not hidden behind the word “certified.”

The prototype’s retained expression is

$$A_N(S) = \sum_{n=1}^{N} a_n n^{-S}. \tag{9}$$

Using a prefix rather than a count of nonzero coefficients makes cancellation and sparse storage unambiguous. A user-facing term goal can be implemented above this layer, but it must not redefine unknown prefix entries as zeros.

5.3 An exact elementary tail bound

Proposition 5.1 (Tail from a coefficient majorant)**.** *Assume* (8)*, let* $M = N + 1$*, and let* $S > p + 1$*. Then the Dirichlet series converges absolutely and*

$$|A(S) - A_N(S)| \le C M^{p-S} \left(1 + \frac{M}{S - p - 1}\right). \tag{10}$$

Proof. Put $t = S - p > 1$. Absolute values reduce the omitted tail to $C \sum_{n=M}^{\infty} n^{-t}$. The decreasing summand is bounded by its first term plus the integral from M to infinity. Evaluating that integral gives $CM^{-t}(1 + M/(t - 1))$, which is (10). □

This is the same elementary comparison principle used by the existing Zeta adapter, now applied to a transported coefficient-growth bound. The incremental point is the closed coefficient-bound calculus below, not the integral test itself. At integer $S > p + 1$, every quantity returned by the prototype’s `tail_bound` method is rational and computed exactly.

5.4 Addition and scalar multiplication

For two coefficient sequences with bounds (C_a, p_a) and (C_b, p_b), addition admits

$$C_{a+b} = C_a + C_b, \qquad p_{a+b} = \max(p_a, p_b). \tag{11}$$

Scalar multiplication by a rational r admits $(|r|C_a, p_a)$. The known prefix of a binary operation is the intersection of the input prefixes. These rules are conservative when operands are correlated: canceling retained coefficients does not, by itself, cancel independently declared unknown tails.

5.5 Multiplication is Dirichlet convolution

Proposition 5.2 (Coefficient and majorant transport)**.** *In a common half-line of absolute convergence, the product of two series* (6) *has coefficients*

$$c_n = \sum_{d \mid n} a_d b_{n/d}. \tag{12}$$

If the input sequences obey their declared bounds, a valid elementary output bound is

$$|c_n| \le C_a C_b n^{\max(p_a, p_b)+1}. \tag{13}$$

All output coefficients through $N = \min(N_a, N_b)$ *are determined by the two input prefixes.*

Proof. Absolute convergence permits regrouping the product by $n = dm$, which gives (12). Every divisor factor satisfies $d^{p_a}(n/d)^{p_b} \le n^{\max(p_a,p_b)}$. There are at most n positive divisors of n, giving the stated bound. For $n \le N$, every divisor and complementary divisor is at most N, so no unknown coefficient is used. □

The divisor-count estimate is intentionally loose. It keeps p integral and all subsequent majorants rational at integer S. A sharper estimate such as $\tau(n) \le 2\sqrt{n}$ would reduce the growth exponent at the price of a larger exponent domain in the implementation. Nothing in the prototype claims an optimal bound or an optimal abscissa of convergence.

For dense known prefixes, the retained pair count is exactly

$$H_N = \sum_{d=1}^{N} \left\lfloor \frac{N}{d} \right\rfloor. \tag{14}$$

The elementary harmonic-sum comparison gives $H_N \le N(1 + \log N)$. A row-major loop therefore stops its inner traversal at $m \le N/d$ instead of forming an N by N product and discarding most pairs afterwards. Sparse input support reduces the actual work further.

5.6 Reciprocals with a coefficient bound, not only a formal recurrence

A reciprocal prefix can be computed whenever $a_1 \neq 0$:

$$b_1 = \frac{1}{a_1}, \qquad b_n = -\frac{1}{a_1} \sum_{\substack{d \mid n \\ d \ge 2}} a_d b_{n/d} \quad (n > 1). \tag{15}$$

The missing ingredient in a purely formal implementation would be a useful global bound for the new sequence. The following elementary construction supplies one.

Proposition 5.3 (A computable reciprocal majorant)**.** *Suppose $a_1 \neq 0$ and (8) holds. Choose an integer $q > p + 1$ for which*

$$\kappa = \frac{C}{|a_1|} 2^{p-q} \left(1 + \frac{2}{q - p - 1}\right) < 1. \tag{16}$$

The sequence defined by (15) satisfies

$$|b_n| \le \frac{n^q}{|a_1|} \quad (n \ge 1). \tag{17}$$

Its absolutely convergent Dirichlet series represents $1/A(S)$ for $S > q + 1$.

Proof. Set $D = 1/|a_1|$. The base coefficient satisfies $|b_1| = D$. If the bound holds for all indices below n, then each n/d in (15) is smaller than n, and hence

$$\begin{aligned} |b_n| &\le D n^q \frac{1}{|a_1|} \sum_{\substack{d \mid n \\ d \ge 2}} |a_d| d^{-q} \\ &\le D n^q \frac{C}{|a_1|} \sum_{d=2}^{\infty} d^{p-q} \le D n^q \kappa < D n^q. \end{aligned}$$

The final bound on the sum is the first-term-plus-integral estimate beginning at two. This proves the coefficient inequality by strong induction. For $S > q + 1$, both the original and reciprocal series converge absolutely. Their product has coefficient one at index one and zero at every larger index by (15). Thus their product is one, proving the reciprocal claim. □

The expression in (16) tends to zero as q increases, so a suitable integer exists. With rational C and a_1, the search can compare κ to one using exact fractions. The implementation bounds both this search and the coefficient work. Budget exhaustion raises an explicit exception instead of returning an unproved reciprocal bound.

A sieve-style evaluation avoids asking for divisors separately at every index. After coefficient b_m is known, its contributions $a_d b_m$ are accumulated at indices $dm \le N$, $d \ge 2$. All contributions required for the next coefficient have arrived from smaller indices. This is the same retained-pair geometry as (14).

5.7 Executable examples

```
from dirichlet_jet import DirichletJet

z = DirichletJet.zeta(128)
z2 = z.multiply(z)
reciprocal = z.reciprocal()
identity = z.multiply(reciprocal)

assert dict(identity.coefficients) == {1: 1}
assert z2.coefficient(12) == 6
assert reciprocal.coefficient(6) == 1
assert reciprocal.coefficient(12) == 0

finite_value = reciprocal.evaluate_prefix(10)
absolute_tail = reciprocal.tail_bound(10)  # exact Fraction
```

For ζ^2, the coefficients count the divisors of n. For the reciprocal of Zeta, the computed prefix agrees with an independent elementary prime-factorization implementation of the Möbius function. These are useful structural oracles, not comparisons with a second transcription of the same convolution loop.

The example does not assert that the conservative reciprocal majorant is the sharp bound for Möbius coefficients. A source-aware specialized factory could provide a stronger bound, but the implemented reciprocal operation intentionally uses only its input's declared coefficient majorant.

6 Performance evidence and implementation tradeoffs

6.1 What was actually measured

The independent Python convolution prototype was measured on dense Zeta prefixes. Input construction was outside each timed interval. Each size was repeated five times; the table reports median elapsed seconds in this session. These are not Wolfram or Mathics timings, and no speedup over the existing package has been measured.

Known prefix N	Retained pairs H_N	Unfiltered N^2 pairs	Median seconds
64	280	4,096	0.00047
256	1,466	65,536	0.00216
1,024	7,262	1,048,576	0.01086
4,096	34,720	16,777,216	0.04985

The exact pair counts explain the loop's search geometry independently of the timer. The implementation still pays for fraction arithmetic, dictionary updates, output normalization, and prefix-bound checks. The measurement

does not estimate peak memory, simplifier costs in Wolfram, or the behavior of a symbolic coefficient ring. The complete platform string and individual summary values are in `evidence/python-benchmark.json`; the benchmark script is included.

6.2 Why integer indexing is useful even without a speed claim

Integer indexing removes a particular proof obligation from this family: recognizing equality or order between sums of weights that are known to be logarithms of integer products. A coefficient of 6^{-S} produced from $2^{-S}3^{-S}$ has index six immediately. There is no need to ask a generic symbolic simplifier to recognize $\log 2 + \log 3 = \log 6$.

This observation justifies a data representation, not a blanket recommendation to replace the existing real-weight implementation. Irrational-power germs, Fourier-polynomial coefficients, and the package's other scales require their own contracts. The integer representation should expose a faithful conversion back to the existing $w^{\log n}$ rows rather than silently assuming all real weights have integer logarithmic indices.

6.3 What the prototype intentionally does not implement

The Python class does not integrate with `GeneralizedSeries`; it is an independent reference model. It does not implement arbitrary real or complex coefficients, noninteger phase evaluation, general phase composition, parameter-uniform estimates, automatic source recognition, or exact cancellation of correlated unknown tails. Its finite-polynomial factory also uses the generic conservative tail interface, rather than exploiting finite support to report a zero tail through a second exactness channel.

Those are explicit prototype boundaries. They are not additional defects attributed to the upstream repository. The implementation is small enough to serve as an oracle for a future family-specific module and to test whether integer indexing is worth integrating before complicating the public result schema.

7 Wolfram 15 and Mathics: a targeted compatibility assessment

7.1 What the successful Wolfram run establishes

The actual native kernel identified itself as

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

It loaded the complete imported standalone and executed the public witness calls. The term-goal candidate also executed there. Therefore N01 and N02 are not merely predictions based on a different Wolfram version, a Mathics emulation, or an independent Python reimplementation.

The no-goal controls are especially important for N02: the candidate must not change the series selected by the cutoff alone. The observed unchanged Zeta and Lerch expressions demonstrate that limited property for the two chosen inputs. They do not establish every parameter case or an end-to-end release gate.

7.2 What source inspection says about Mathics

The repository isolates several host differences behind Mathics-specific helpers: assumption/sign reasoning, simplification, Taylor expansion, numerical evaluation, inverse-branch reasoning, and compatibility shims. The source contains explicit guards around native behaviors that the package does not treat as interchangeable with Wolfram. This architecture matters when assessing a proposed edit: a fix should use the package's

existing exact-order and validation helpers rather than add an unreviewed dependency on a native symbolic oracle. [11, 12, 13]

The term-goal patch changes only Boolean conditions, integer bounds, and the existing `less` comparison calls. It does not introduce a new special function, a native-only calculus operation, or a new root finder. Both affected constructors are shared canonical source, so the control-flow issue is not inherently Wolfram-specific. Whether every corresponding public witness reaches that path and produces the same output under a particular Mathics version still requires an actual run.

The affine prototype similarly reuses the package's public arithmetic, but its complete loader/option/pattern behavior has not been validated under Mathics. Its code is intended to be straightforward to test, not presented as a compatibility-certified drop-in. Mathics is an independent implementation of the Wolfram Language, so successful Wolfram execution cannot substitute for that second run. [14]

7.3 A focused cross-host acceptance set

For the proposed changes, the useful acceptance cases are narrowly defined. The bare-atom controls must still succeed; the combined goal/cutoff cases must retain the same selected blocks in both hosts; a no-goal request must remain unchanged; and the large-cutoff, one-term Zeta request must no longer fail solely because of the inactive work implied by the cutoff.

Boundary cases should additionally cover an exact Zeta cutoff at $\log m$, a goal larger than the number of available finite Lerch blocks, an invalid goal, a goal larger than the budget but an earlier cutoff, and an affine translation whose weight-zero block cancels. These cases follow from the specific changes in this article. They are not a restatement of the existing general cross-kernel testing program.

The packaged `RunProbes.wl` uses plain records rather than relying on a shared implementation of `TestReport`. It performs no download or repository mutation. It is a proposed reproduction driver; the successful observations in this review came from separately issued focused calls, not from execution of the complete packaged file.

8 Concrete integration and development sequence

8.1 First: land the bounded request fix independently

Stage the three-edit canonical change, rebuild the standalone using the repository's own generation path, and run the focused N02 comparisons on both hosts. Keep this change independent of affine lifting and the integer-indexed prototype. Its coefficient formulas and result schema need not change.

The acceptance condition is not merely that the normal expressions look shorter. Returned counts, first omitted terms, remainder powers, and explicit bounds must all correspond to the selected shorter prefix. The native candidate already demonstrates the expected frontier change for the selected examples.

8.2 Second: admit a small affine grammar at the constructor boundary

Use the rational-affine prototype as a behavioral sketch, not as an instruction to call a public constructor recursively on every subtree. The integrated path should identify one admitted atom, prove the permitted coefficient conditions, construct that atom in its own coordinate, apply the exact affine operations, and finish the request once on the normalized result.

Its diagnostic should distinguish an unsupported *wrapper grammar* from an unsupported special function. For example, a rejected variable-dependent multiplier should say that the affine defining-sum route requires constant

coefficients. A later native fallback may still be attempted under the user's chosen backend policy, but its incidental coefficient error should not be the only explanation available when the special leaf itself is recognized.

8.3 Third: evaluate integer-indexed algebra as a separate feature

A small initial Wolfram-facing interface could construct an integer-indexed Dirichlet prefix, combine prefixes by addition and convolution, compute an admitted reciprocal, and export $\{\log n, a_n\}$ rows plus a transported majorant. It should first support a single common phase S and rational coefficients, matching the independent oracle.

Only after that core agrees with the independent tests should it widen the coefficient ring. Differentiation in S is a particularly natural next extension because

$$\frac{d^k}{dS^k} n^{-S} = (-\log n)^k n^{-S}. \tag{18}$$

A simple bound $(\log n)^k \le n^k$ for $n \ge 1$ would preserve an elementary integer growth exponent, though it is conservative. For Zeta, the corresponding termwise derivative formulas in the absolute-convergence half-plane are also recorded in DLMF. [16]

This proposed extension is tied to the integer-indexed family and its coefficient-majorant proof. It is not a proposal to claim differentiability from an arbitrary asymptotic remainder. The prototype currently implements neither derivative coefficients nor that extended coefficient ring.

A second controlled extension is an integer affine phase shift: $A(S+b) = \sum a_n n^{-b} n^{-S}$. For integer b, rational coefficients remain rational; for negative b, the coefficient growth exponent increases by $-b$. This can be added without admitting unrelated exponential rates. Arbitrary phase compositions should remain outside this initial feature.

8.4 What should not be coupled to these changes

There is no need to change the entire result schema, replace the general precision calculus, redesign all native dispatch, or promise arbitrary transseries closure to fix N01 and N02. Those wider topics already have their own record. A family-specific implementation should earn its place by reducing an observed failure and agreeing with a small independent algebra, rather than becoming a prerequisite for unrelated repairs.

9 Artifacts, reproduction, and final assessment

9.1 Archive contents

Artifact	Purpose and evidence status
`article/review.tex`, `article/review.pdf`	Source and rendered article.
`code/patch_dirichlet_goals.py`	Guarded staging utility for the three-edit candidate; synthetic anchor tests passed, and the corresponding edits passed five native controls.
`code/AffineDirichletExpansion.wl`	Restricted additive/scalar lifting prototype; complete helper execution remains unverified.

Artifact	Purpose and evidence status
code/RunProbes.wl	Local-file-only Wolfram/Mathics reproduction driver; full packaged driver not executed here.
code/dirichlet_jet.py	Exact-rational integer-indexed prefix, convolution, reciprocal, and coefficient-bound implementation.
code/benchmark_dirichlet.py	Reproducible benchmark of that independent Python prototype only.
tests/test_dirichlet_jet.py	32 passing independent test methods, including four synthetic patch-fixture methods.
evidence/	Native transcriptions, source coverage, novelty ledger, independent status, complete Python test output, and prototype benchmark results.

9.2 Running the independent checks

From the archive root:

```
python -m unittest discover -s tests -v
python code/benchmark_dirichlet.py
```

The tests use only the standard library. They check divisor-count coefficients, a prime-factorization Möbius oracle, reciprocal identities, random exact convolutions, prefix boundaries, rational majorants, input validation, immutability, and explicit work budgets. Finite partial-tail comparisons are sanity checks accompanying the displayed infinite-tail proofs, not substitutes for those proofs.

To stage the candidate without altering the source checkout:

```
python code/patch_dirichlet_goals.py /path/to/Asymptotic /path/to/stage
```

The output directory must not contain the staged artifacts already. A changed or already-patched source is rejected by the anchor checks. Apply the resulting canonical change in a disposable checkout and use the upstream builder before comparing the generated standalone. This review does not supply a replacement copy of the entire upstream repository.

For a fresh Wolfram or Mathics session, set a local path before loading the probe driver:

```
AuditPackagePath = "/path/to/Asymptotic/AsymptoticAnalysis.wl";
AuditOutputPath = "/path/to/observations.wl";
Get["/path/to/audit/code/RunProbes.wl"];
```

Use a separate fresh process for the baseline and candidate. The optional output path is a plain-text observation record, not an acceptance receipt.

9.3 Assessment

The new actionable defects are small and localized: a missing affine admission route, and a conditional that treats two simultaneous constraints as alternatives. Their importance is not that they undermine the package's entire

mathematics. They make capabilities already present in the repository unexpectedly hard to use and make request behavior depend on the selected special-function path.

The term-goal candidate is the most immediately supported change: it was executed against the pinned standalone and preserves the selected no-goal controls. The affine helper is a restricted implementation sketch with successful underlying arithmetic controls, not a fully validated integration. The integer-indexed Dirichlet prototype is a separate, implemented development option with a precise algebra and explicit coefficient-bound assumptions.

This is the full incremental conclusion supported by the source inspection and executions obtained here. It does not manufacture additional findings from already catalogued issues, transport failures, or features outside an explicitly admitted mathematical contract.

References

[1] Asymptotic Contributors. *External code-review index and retirement records*, pinned revision. Repository review index.

[2] Asymptotic Contributors. *Code review status and implementation register*, pinned revision. Maintained register.

[3] Asymptotic Contributors. *Wave-3 review index*. Wave-3 packages and finding summaries.

[4] Asymptotic Contributors. *Wave-5 review index*. Wave-5 packages and finding summaries.

[5] *Incremental technical audit*, retained report 24, especially the reciprocal-square Lerch discussion. Original article source.

[6] Asymptotic Contributors. *AsymptoticAnalysis.wl*, canonical kernel, especially `localCoordinate`, `forwardCore`, and `makeForwardObject`. Pinned canonical source; relevant request/object assembly begins near lines 610–740.

[7] Asymptotic Contributors. *DirichletSpecialFunctions.wl*, especially `dirichletSpecialMake`, `dirichletZetaForward`, `dirichletLerchForward`, and the final whole-expression dispatcher. Pinned defining-sum module.

[8] Asymptotic Contributors. *SeriesArithmetic.wl*, regular operands and held arithmetic dispatch. Pinned source, especially the first 180 lines.

[9] Asymptotic Contributors. *SeriesOperations.wl*, ordinary series data, operations, and result construction. Pinned source.

[10] Asymptotic Contributors. *SeriesEnvelopeArithmetic.wl*, compatible approaches and conservative composite arithmetic. Pinned source.

[11] Asymptotic Contributors. *MathicsAssumptions.wl*, finite-real and sign consequence helpers. Pinned source.

[12] Asymptotic Contributors. *MathicsSimplification.wl*, host simplification and limit adapters. Pinned source.

[13] Asymptotic Contributors. *MathicsCompatibility.wl*, bounded compatibility helpers. Pinned source.

[14] Mathics3 Project. *Mathics3*. Official project site, consulted 10 September 2026. `https://mathics.org/`.

[15] Wolfram Research. *SeriesTermGoal*. Wolfram Language documentation, consulted 10 September 2026. `https://reference.wolfram.com/language/ref/SeriesTermGoal.html`.

[16] NIST Digital Library of Mathematical Functions. *Section 25.2: Definition and expansions*, especially equations 25.2.1 and 25.2.7. `https://dlmf.nist.gov/25.2`.

[17] NIST Digital Library of Mathematical Functions. *Section 25.14: Lerch's transcendent*, especially its defining series. `https://dlmf.nist.gov/25.14`.